

COS30002 Portfolio 5: Task 20-B - Custom Project - Practical Plan

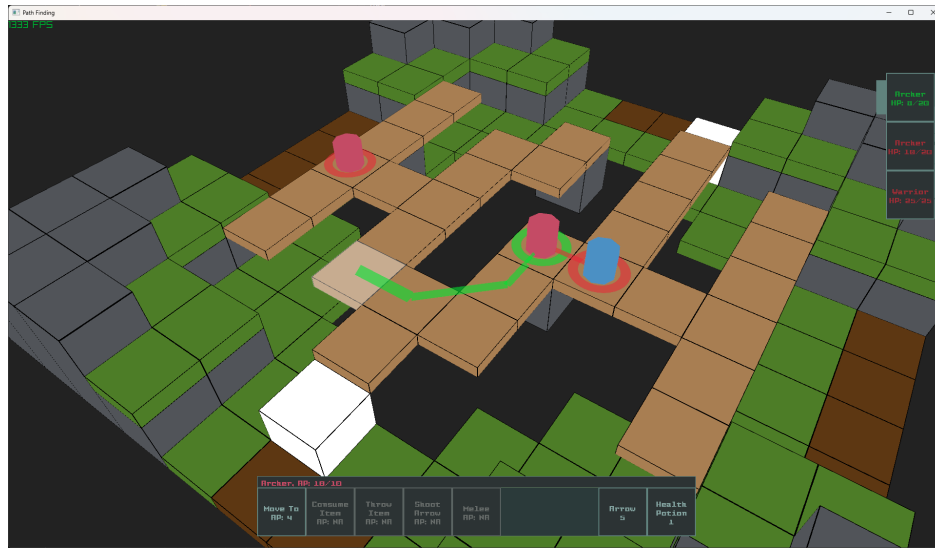
Ewan Robson (103992579@student.swin.edu.au)

June 13 2026

Contents

Introduction	2
ULO Alignment	2
Project Build	3
Option 1: Build with MSVC	3
Option 2: Build with Ninja and Clang (My Preferred choice)	3
Game Design	4
Characters	5
Abilities	5
Items	5
AI	6
Architecture	7
Application [A]	7
Simulation [A]	9
Characters and Turn Order [A][S]	12
Pathing and Goals [A][P][X]	14
Simple Kinematics and Steering [S]	17
Goal-Oriented Action Planning Architecture [A][G][P]	18
Visual Debugger and Measure Success Criteria	19
Navigation and Pathfinding Validation	19
Character State and Combat Bounds	19
Reflection and Integration	20
Task 19: Initial Architecture and Foundation	20
Task 20: Final Integration and Scope Management Reflection	20

Introduction



The document serves to provide all artifacts produced when creating this project such as class UML diagrams, code snippets and a mini game design. The link to the codebase is [linked here](#) and the README details how to build the project in detail using 2 different methods. Prebuilt binaries are provided within the bin folder as an alternative option.

ULO Alignment

[A]: Dev/Architecture Techniques, [G]: Graphs and Searches, [S]: Steering, [P]: Plans and Goals, [X]: Combined Techniques

- [A] **World representation:** 3D tilemap structured in layers, parsed using a custom TOML-inspired file format.
- [G] **Navigation Graphs:** Sparse graphs with support for diagonal connections.
- [G][P][X] **Pathfinding:** A* Pathfinding utilising influence maps to find optimal routes.
- [S][G][X] **Simple Kinematics:** Smoothly follow a path using simple physics.
- [P][X] **Influence Maps:** Hazardous Maps (for mud, poison, barrel radii) and Elevation Maps to guide AI positioning.
- [P][X] **Decision Making:** Goal-Oriented Action Plan (GOAP) allowing the AI to formulate multi-step plans within a single turn, supported by a World State Blackboard.

Project Build

Due to the project being written in C++, there are two options for building on **Windows Only** (*Linux/Mac has not been tested, do not expect to build/run when using Ninja/Clang on those platforms*). Option 1; building with MSVC which is the easiest, or Option 2; using Ninja and Clang.

Option 1: Build with MSVC

- Install [CMake](#).
- Install [Visual Studio](#) enabling C++ dependencies during the installing process.
- **Generate solution:**
 - Open terminal in project root (`../cos30002-103992579/Custom Project`).
 - Run script `gen_msvc_project.bat`.
 - * *Initial run fetches dependencies which may take some time.*
 - * Alternatively run `cmake -S . -B build -G "Visual Studio 17 2022" -A x64`, which the bat script calls anyways.
- **Build:**
 - Open `./build/crpg-combat.sln` in Visual Studio.
 - Build solution by pressing Ctrl+Shift+B.
- **Run:**
 - Right-click `crpg-combat` project in Solution Explorer.
 - Click “Set as Startup Project”.
 - Execute.
 - * F5: Run within editor
 - * Open Terminal and execute `./build/Debug/crpg-combat.exe` from within the root

Option 2: Build with Ninja and Clang (My Preferred choice)

- Install [CMake](#).
- Install [LLVM/Clang](#).
- Install [Ninja](#).
 - Can use any other generator that CMake supports.
- Add CMake, Ninja, and Clang to system PATH.
- **Generate build files:**
 - Open terminal in project root (`../cos30002-103992579/Custom Project`).
 - Run `gen_ninja_clang.bat`.
 - * *Initial run fetches dependencies which may take some time.*
 - * Alternatively run `cmake -S . -B build -G "Ninja" -D CMAKE_C_COMPILER=clang -D CMAKE_CXX_COMPILER=clang++`, which the bat script calls anyways.
 - * If you want lsp support append `-DCMAKE_EXPORT_COMPILE_COMMANDS=ON` to generate `compile_commands.json` and point lsp it from the build folder.
- **Build:**
 - Run `cmake --build build`.
- **Run:**
 - Execute `./build/crpg-combat.exe`.

Game Design

A heavily scoped down, CRPG, [Divinity Original Sin 2](#) inspired combat system. Ever since I played through [Baldur's Gate 3](#) I have been fascinated by the AI of this game, and I'd love to try and create a similar experience.

Turn-based combat creates endless possibilities for the AI's behaviour. The most interesting aspects I found are how the enemies interact with the environment (finding high ground for increased damage, or creating hazardous ground while also avoiding it). Another behaviour I'm interested in is the AI cooperating with its allies, such as throwing a healing potion if their health is low or staying close to each other.

See fig. 1 and fig. 2 below which demonstrates a single frame:

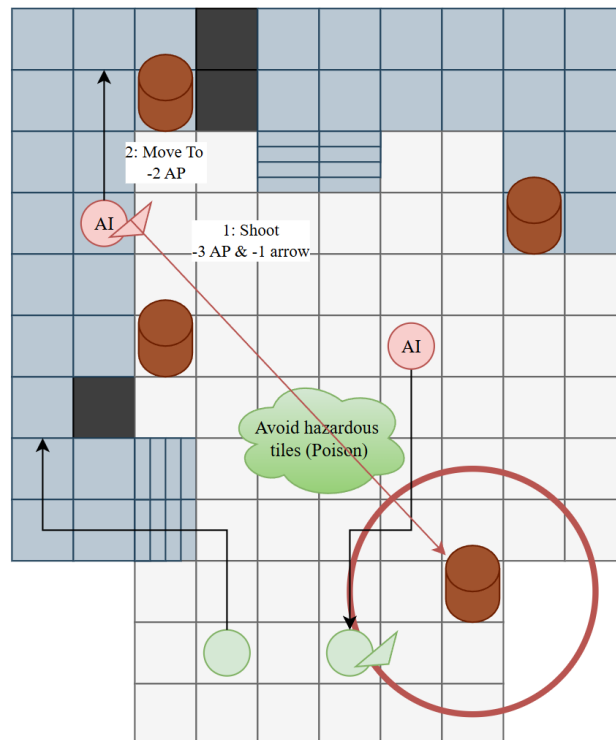


Figure 1: World representation with the enemy archer (AI) performing their turn. Firstly shoots the barrel as it's close to one of their opponents, before moving further away from player as it's weak up close.

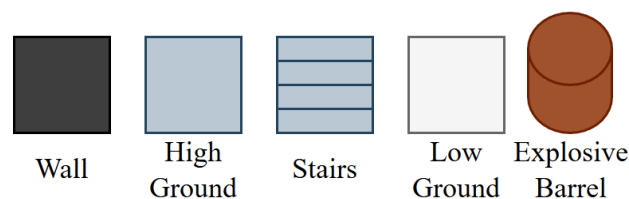


Figure 2: Legend for Figure fig. 1

Characters

Characters are not split into traditional classes like a Warrior or an Archer; each will contain a mix of different abilities and a set number of Action Points (AP) assigned through attached components. Each ability requires AP to perform (some may also require an item, such as an arrow for shooting). This allows the AI to perform more than one action per turn as long as its AP bank allows for it. After finishing their turn, the next player's turn starts, refreshing their AP.

The following list of abilities and items will allow for diverse decision making when combined:

Abilities

All calculated value outcomes are scaled through a random percentage of 0-100%. $x < 10\%$ results in a miss.

Ability Name	Cost
Move to	1 AP per metre
Melee	5 (Archer), 4 (Warrior) AP
Shoot Arrow	4 AP and 1 Arrow
Throw Item	1 AP and Item in Inventory
Consume Item	1 AP and Item in Inventory

Items

All calculated value outcomes are scaled through a clamped random percentage of 40-100%. *Slime Bottle is probably out of scope.*

Item name	Description
Healing Potion	Increases health by 50%
Poison Potion	Every turn take 5 Damage for 3 turns
Slime Bottle	+1 AP per meter when moving through

AI

To facilitate autonomous decision-making without the scalability limitations and rigidity of traditional Finite State Machines (FSMs), the agent architecture utilise Goal-Oriented Action Planning (GOAP). This decoupling of goals from the specific sequences of actions allows the agent to dynamically adapt to the environmental conditions and create believable emergent behaviour.

The agent's decision-making loop is driven by a set of competing Goals, each assigned an Insistence value that dynamically updates based on internal state variables and the environment. The goal with the highest current insistence is selected as the active planning target.

Self	Self Inventory	Current Target	Per Ally
HP: int AP: int	HP: int Poison: int Arrow: int	HP: int DistBTW: float InRange: bool InBarrelRange: bool	HP: int DistBTW: float InRange: bool

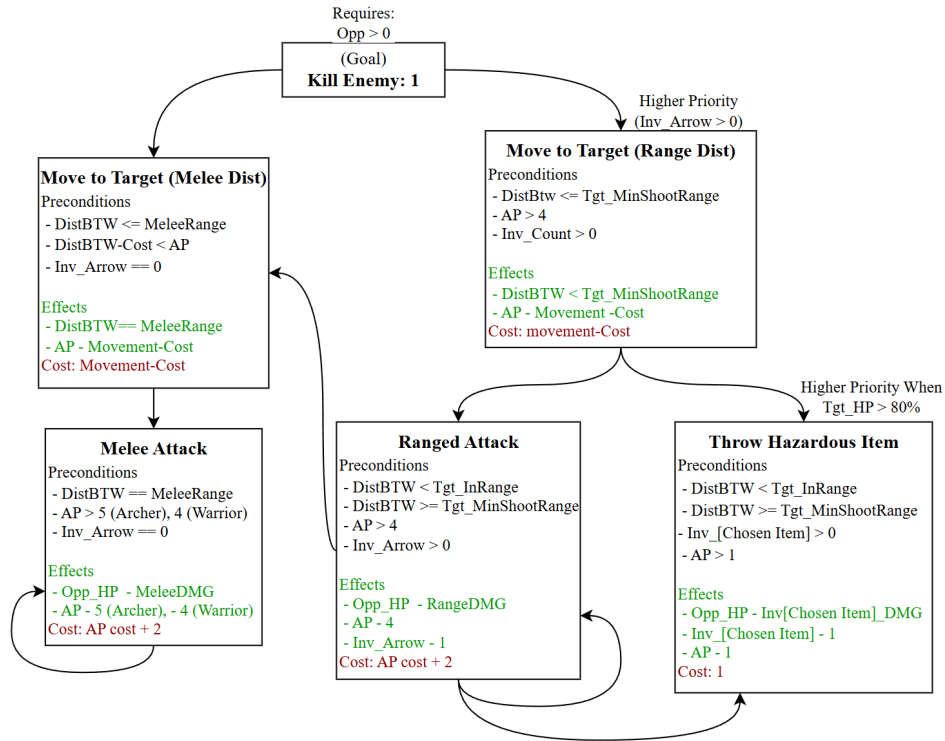


Figure 3: GOAP attacking goals and related actions. Evaluated based on target proximity, line of sight, and offensive resource availability.

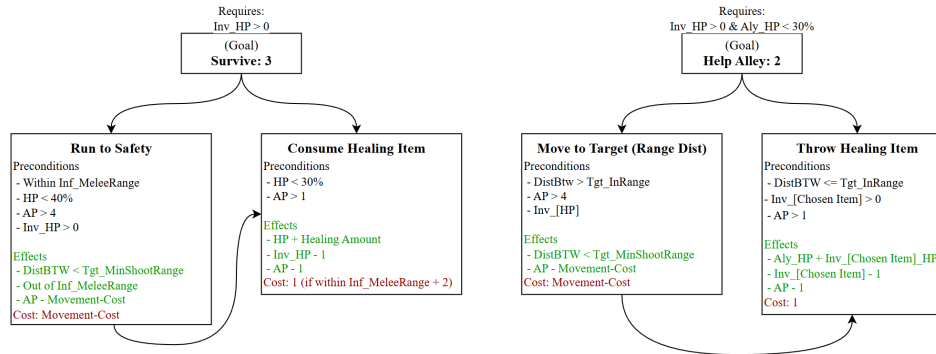


Figure 4: GOAP sustain goals and related actions. Evaluated primarily against health thresholds, defensive positioning, and resource scarcity. When survival metrics drop below critical levels, the insistence for safety or replenishment exponentially overrides offensive desires.

Architecture

Application [A]

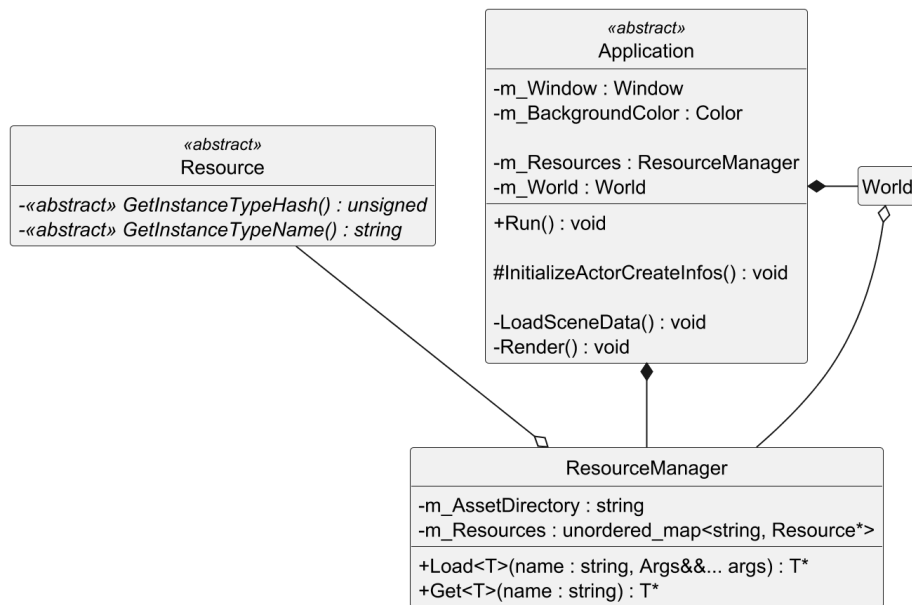


Figure 5: Application Facade Class UML

The **Application** class acts as the main facade and serves as an abstract base class from which user-defined applications, such as the specialised **Game** implementation, inherit. This core system manages and links essential architectural components, including the window, the world state, and the resource manager. It is also responsible for driving the main runtime loop, updating the world, and dispatching render calls.

```

// Main runtime loop handled by the abstract Application class
float lastTime = 0.0f;
while (m_Window.IsOpen())
{
    // 1. Calculate the delta time between frames for smooth simulation
    float time      = static_cast<float>(GetTime());
    float deltaTime = time - lastTime;
    lastTime       = time;

    // 2. Update tick to all active actors in the world
    m_World.CallActorsOnUpdate(deltaTime);

    // 3. Dispatch draw commands to the render queue
    Render();
}

```

The `Game` subclass registers actor creation information during initialisation. This initialisation data is routed to the `CreateActorRegistry`, which is responsible for safely instantiating actors at runtime or during startup by parsing the level data.

```

class Game : public eng::Application
{
public:
    static Game CreateInstance(int argc, char** argv); // Called in Main.cc

    Game(const eng::ApplicationCreateInfo& createInfo);

protected:
    void InitializeActorCreateInfos(eng::World& world) override;
    void OnAfterLoadingLevelData(eng::World& world) override;
};

```

Code for registering actor create infos using the custom `TypeInfo` compile time reflection system

```

// The Game instance registers all possible actor types at startup
void Game::InitializeActorCreateInfos(eng::World& world)
{
    // Links the C++ class to a string tag used by the level data parser
    world.AddSpawnInfo<CameraController>();
    world.AddSpawnInfo<TurnManager>();
    // ...
}

```


Simulation [A]

The **World** class acts as a composite container managing all **Actor** instances. The **Actor** is an abstract base class inherited by any entity that exists within the environment and requires regular updating via an **OnUpdate()** method, including characters, the turn manager, and the tilemap itself.

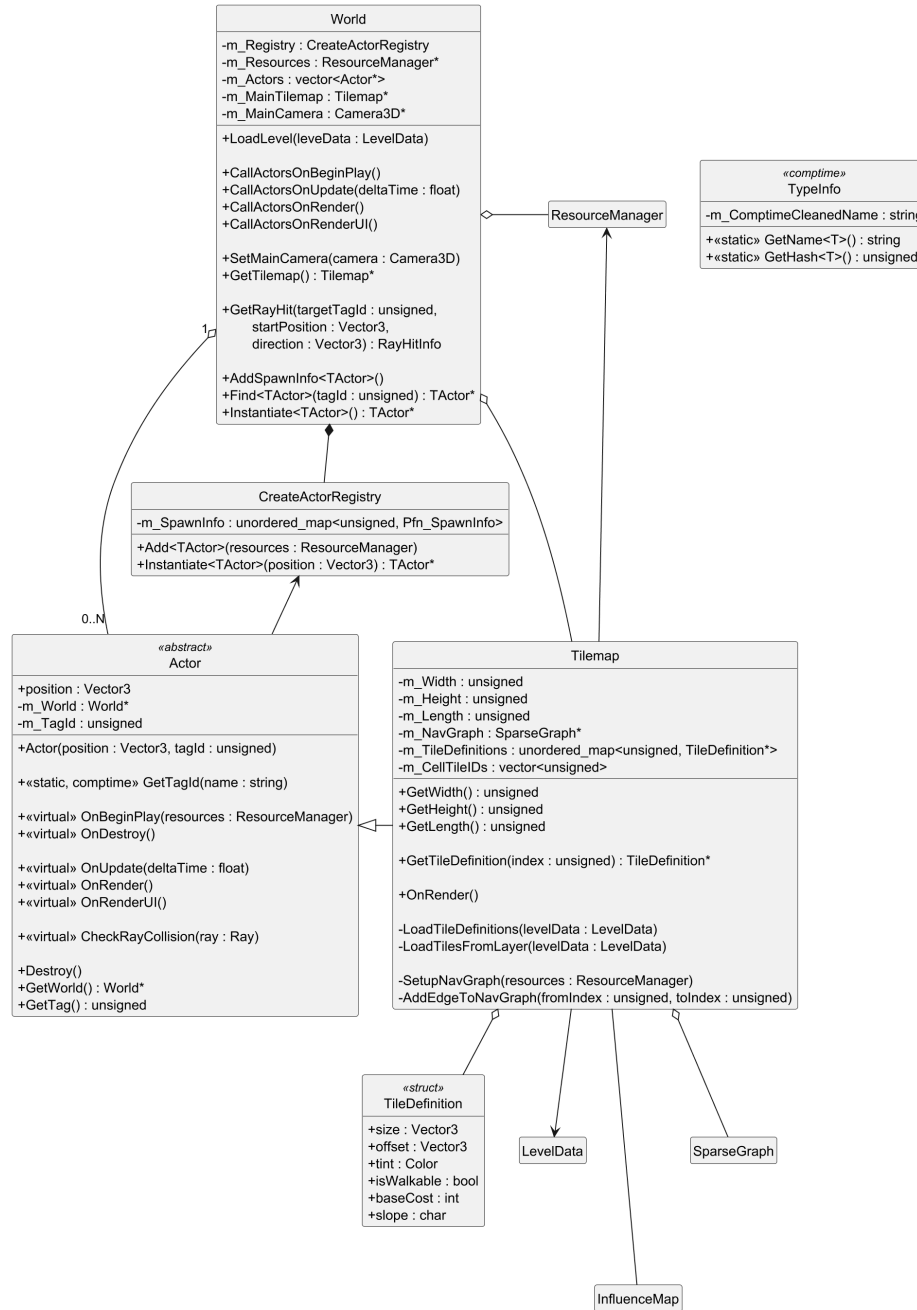


Figure 6: Simulation Layer Class UML

Upon creation, actors receive a reference to the **ResourceManager**, allowing them to readily access shared assets such as 3D models and textures. **Actor** instantiation heavily relies on a **TypeInfo** system for compile-time reflection to register and identify classes cleanly.

```

// Spawns an instance of actor with hash code. Spawn functions are defined at program startup.
// Hash code should be identical to the Actor class TypeInfo<T>::GetHash()
std::unique_ptr<Actor> Instantiate(unsigned hash, const Vector3& position,
                                   const std::string_view& actorName = "Actor")
{
    ...
    Actor::Pfn_SpawnInstance spawnFunc = m_SpawnInfos.at(hash);
    return spawnFunc(position);
}

```

The Tilemap is a specialised actor instantiated by defining the tilemapClass field within the level data configuration. While a standard tilemap does not account for movement penalties, derived classes like MoveWeightedTilemap implement custom logic to load weighted influence maps, effectively adjusting pathfinding costs based on specific terrain configurations.

MoveWeightedTilemap implementation of loading the weighed influence map and storing it within the ResourceManager:

```

// MoveWeightedTilemap maps grid cells to a movement cost influence map
auto* moveWeightCost = resources.Load<eng::InfluenceMap>(
    InfluenceMapName_MoveCostWeight, GetWidth(), GetHeight(), GetLength());

IterateCells(
    [&](const eng::TileIterCellContext& cell)
    {
        // Extract the movement cost (mCost) defined in the TOML file, defaulting to 1
        const eng::LevelTypeDef& levelTileDef = levelData.tileDefinitions.at(cell.id);
        int weight = levelTileDef.GetField<int>("mCost", 1);

        // Inject this cost into the influence map for the A* algorithm to read later
        moveWeightCost->SetSingleCellInfluence(cell.index, weight);
    });

```

The core level layout and entity placement are defined using a custom TOML format. This configuration file maps simple ASCII characters to define tile definitions and actor parameters, allowing for iteration of the game world without recompiling C++ code.

```
[Meta]
name = "Test Level"
...
tilemapClass = "game::MoveWeightedTilemap"

# ID = "TileName" | field=value, field=value, ...
[TerrainDefs]
+ = "Floor"      | tint = "c8bfba", walkable = true, height = 0.5
W = "HighWall"   | tint = "515459", height = 3.0
w = "ShortWall"  | tint = "515459", height = 1.0
v = "Stairs"     | tint = "c8bfba", walkable = true, slope = 's'

# ID = "ActorClass" | field=value, field=value, ...
[ActorDefs]
B = "game::Barrel" | height = 0.5, damage = 10
a = "game::Character" | tint = "c44b65", tm = 0, ty = "Archer", dex = 6.0, HP = 3, AR = 5
w = "game::Character" | tint = "4b90c4", tm = 1, ty = "Warrior", maxHP = 20, HP = 1

[Terrain:0] # [Header name : layer (Height starting position)]
. + + + .
. + v + .
+ + + + +
+ W w W +
+ + + + +
[Actors:0]
. . . . .
. . . . .
w . . . B
. . . . .
. . a . .
```

Characters and Turn Order [A][S]

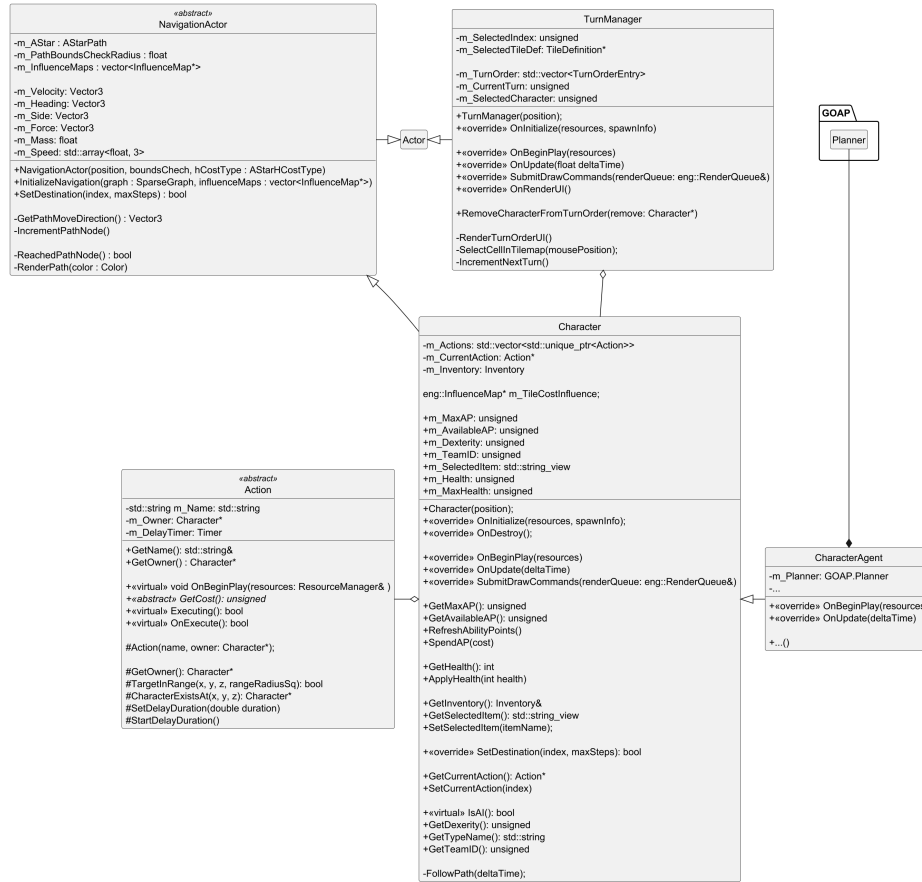


Figure 7: Turn Order and Characters Class UML

The combat loop is orchestrated centrally by the **TurnManager**, which maintains the active sequence of entities using a `std::vector<TurnOrderEntry>`. This manager handles the transition of game states via `IncrementNextTurn()`, sorting characters by their `m_Dexterity` attribute to determine turn order. It is also responsible for maintaining a stable game state by dynamically unregistering defeated entities using `RemoveCharacterFromTurnOrder()`.

Every participant in combat derives from the **Character** base class, which in turn inherits from **NavigationActor** to gain immediate access to the pathfinding and physics-based steering systems [S]. The combat capabilities of each character are strictly governed by an Action Point (AP) economy. Characters track their `m_MaxAP` and `m_AvailableAP`, and must validate their resource pool via `SpendAP()` before committing to a move or attack.

To decouple ability logic from the character state, the architecture employs the Command pattern via the abstract **Action** class.

```

class Action
{
public:
    // ...
    virtual void OnBeginPlay(eng::ResourceManager& resources) {}
    // Defines the Ability Point cost
    virtual unsigned GetCost() = 0;
    // Controls the duration of the action executing
    virtual bool Executing();
    // Must include `Action::OnExecute()` within overridden implementation as it makes the
    //   ↪ character
    // spend the AP to execute this action.
    virtual bool OnExecute();

    // ... Helper functions that are commonly used by most actions such as a timer...

private:
    std::string m_Name;
    Character* m_Owner; // Character Actor that owns this Action within their action list
    eng::Timer m_DelayTimer; // Delay BTW actions, prevents AI from doing > 1 action instantly

```

A **Character** maintains a collection of these **Action** objects (e.g., **Melee**, **Shoot Arrow**, **Move**) stored within an `std::vector<std::unique_ptr<Action>>`. When it is a character's turn, they select an action, set it as the `m_CurrentAction`, and invoke its `OnExecute()` routine. This heavily modularised the codebase; adding a new spell or attack simply requires creating a new class derived from **Action** and implementing its unique `GetCost()` and spatial requirement checks (like `TargetInRange()`), without modifying the core **Character** class.

Furthermore, the architecture enforces strict parity between human players and artificial intelligence. Autonomous enemies are instantiated as **CharacterAgent** objects, a subclass of **Character**.

```

class CharacterAgent : public Character
{
private:
    GOAP::Planner m_Planner;

public:
    void OnUpdate(float deltaTime) override
    {
        // 1. Update GOAP world state
        // 2. Select optimal action from the standard m_Actions pool
        // 3. Execute via standard Character pipeline
    }
};

```

By nesting the `GOAP::Planner` inside the derived **CharacterAgent**, the AI utilises the same AP economy and **NavigationActor** bounds as the player. This saves on development type, prevents duplicate code and aligns the AI's capabilities to the same as the player.

Pathing and Goals [A][P][X]

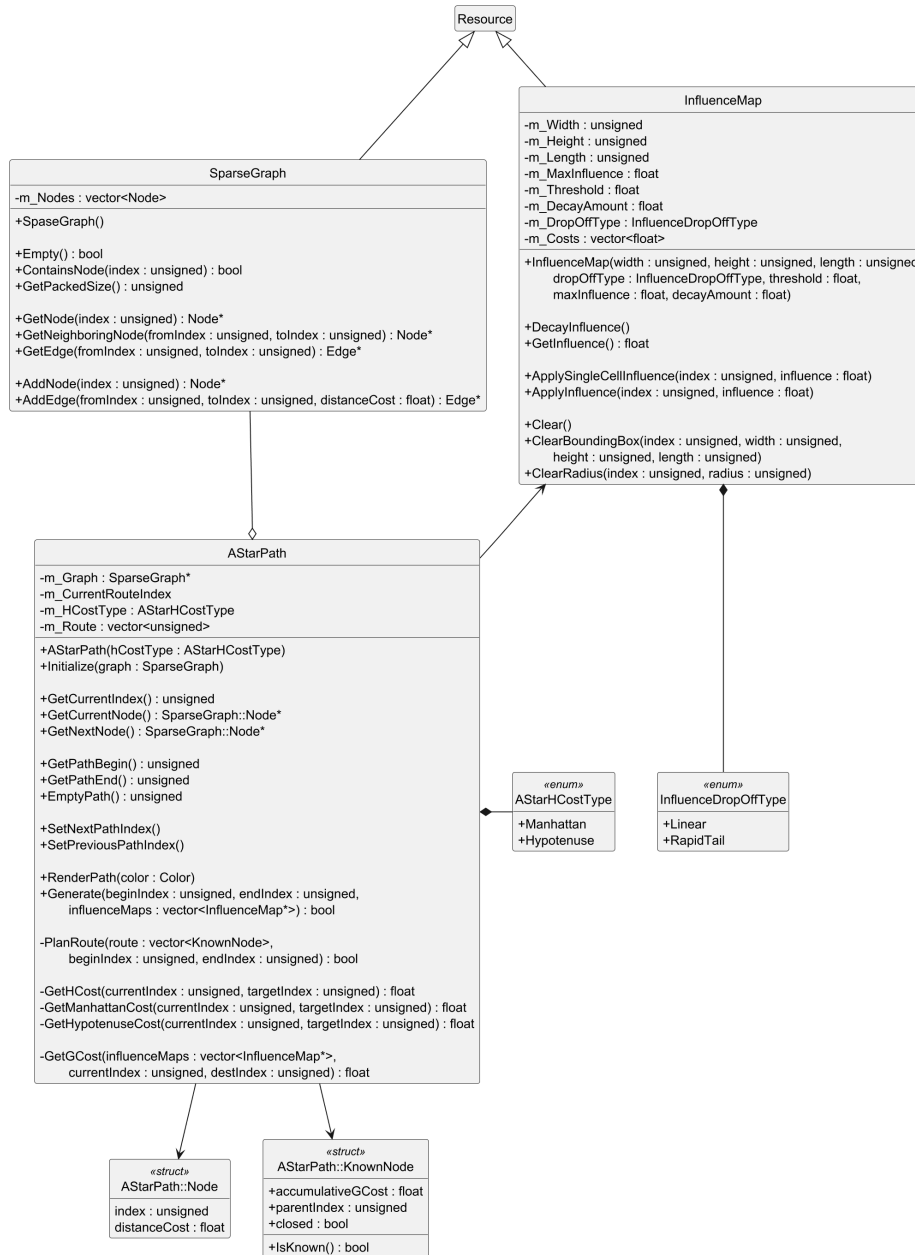


Figure 8: Path Finding Class UML

Navigation across the tilemap is handled by a custom pathfinding system utilised by all autonomous characters. Core to this system is the **SparseGraph**, which employs a contiguous `std::vector` of nodes rather than a dictionary to link graph elements together. Because the **SparseGraph**'s internal array size exactly matches the size of the tilemap, spatial indices map directly to graph nodes, resulting in highly CPU cache-friendly memory access patterns.

During initialisation, the tilemap populates this graph by iterating through all walkable cells and dynamically generating edges to adjacent tiles. The generation logic handles elevation by linking standard tiles to sloped tiles.

```
// Tilemap dynamically wires up the navigation graph during initialisation
void Tilemap::SetupNavGraph(ResourceManager& resources)
{
    m_NavGraph = resources.Load<SparseGraph>("Navigation Graph", m_NonAirCellCount);

    IterateCells(
        [&](const TileIterCellContext& cell)
        {
            if (!cell.def->isWalkable) return;

            // 1. Handle verticality: Check if current tile is a slope and link adjacent
            ↪ elevations
            if (AddEdgeFromSlopeToGraph(cell.def, cell.index)) return;

            // 2. Handle flat ground: Wire edges to the standard 8 surrounding tiles
            AddEdgeToNavGraph(cell.index, cell.y, cell.x + 1, cell.z);           // E
            AddEdgeToNavGraph(cell.index, cell.y, cell.x + 1, cell.z + 1, true); // NE
            // ... (remaining directions omitted for brevity)

        });
}
```

Actual route calculation uses the A* algorithm, taking both heuristic functions and local influence maps into account to determine the most optimal path efficiently due to being cache friendly:

```
bool AStarPath::GeneratePath(unsigned beginIndex, unsigned endIndex,
                             const std::vector<InfluenceMap*>& influenceMaps, unsigned maxSteps)
{
    // ... Standard A* priority queue (open list) and visited array (knownNodes) setup ...
    while (!open.empty())
    {
        // Fail-safe to prevent infinite loops on impossible paths
        if (stepCount >= maxSteps) return false;
        stepCount++;

        Node current = open.top();
        open.pop();
        // ... Standard closed-list check and end-condition check ...

        // Iterate over adjacent tiles provided by our custom SparseGraph
        for (const SparseGraph::Edge& edge : m_Graph->GetNode(current.index)->edges)
        {
            if (!knownNodes[edge.toIndex].closed)
            {
                // Inject influence map data into the standard G-Cost calculation.
                // Forces characters to path around hazards (poison).
                float gCost = knownNodes[current.index].accumulativeGCost +
                             GetGCost(influenceMaps, current.index, edge.toIndex);

                float hCost = GetHCost(edge.toIndex, endIndex);
                float fCost = hCost + gCost;

                // If this new path is cheaper, update the node and push to the queue
                if (gCost < knownNodes[edge.toIndex].accumulativeGCost)
                {
                    open.emplace(Node{edge.toIndex, fCost});
                    knownNodes[edge.toIndex].accumulativeGCost = gCost;
                    knownNodes[edge.toIndex].parentIndex      = current.index;
                }
            }
        }
    }

    return PlanRoute(knownNodes, beginIndex, endIndex);
}
```


Simple Kinematics and Steering [S]

To ensure characters navigate their generated A* paths smoothly, the movement logic relies on a custom steering behaviour system. The Character class handles this by applying physics-based forces to update its velocity and position based on the direction of the next node in its path its following.

During the `OnUpdate()` loop, the `FollowPath()` routine evaluates the character's current position relative to its active path nodes. When traversing between intermediate nodes, the character applies a continuous seek force. Once it nears its final destination, it transitions to an arrive force.

```
// Applies the calculated steering forces, factoring in character mass and maximum speed  
↪ limits  
void Character::ApplyForces(float deltaTime)  
{  
    if (m_Velocity == eng::SparseGraph::Node::Invalid)  
    {  
        m_Velocity = Vector3Zeros;  
        return;  
    }  
  
    m_Velocity += (m_Force / m_Mass) * deltaTime;  
  
    // Truncate velocity to max speed  
    float CurrSpeedSqr = Vector3LengthSqr(m_Velocity);  
    if (CurrSpeedSqr > m_MaxSpeed * m_MaxSpeed)  
    {  
        m_Velocity /= std::sqrt(CurrSpeedSqr);  
        m_Velocity *= m_MaxSpeed;  
    }  
  
    position += m_Velocity * deltaTime;  
}
```

Goal-Oriented Action Planning Architecture [A][G][P]

Goap Actions utilise the Actions defined in the Turn Order Section that each character uses. This allows for modular Action creation that can be controlled through the AI (using the GOAP Planner) or the player.

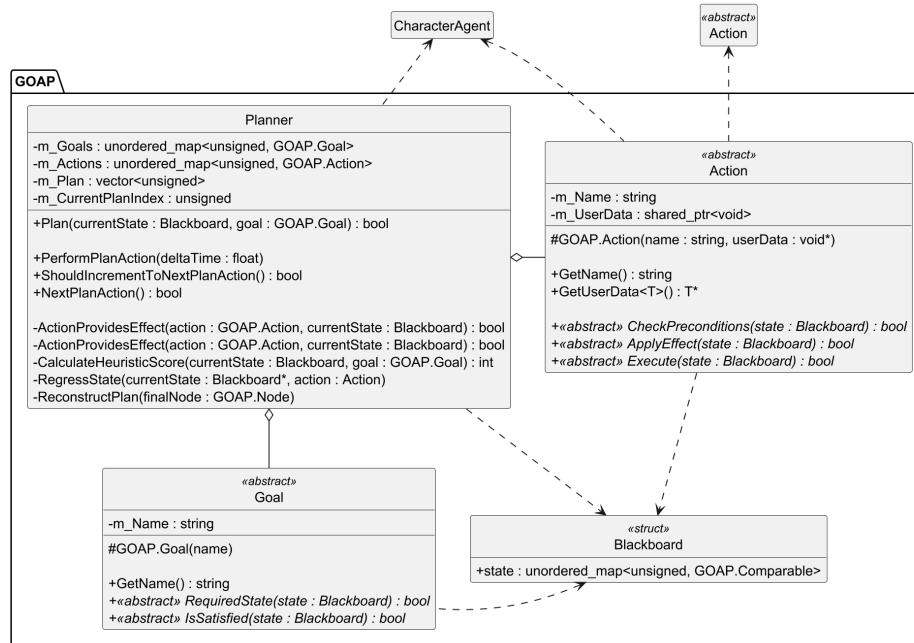


Figure 9: GOAP Planner Class UML

As illustrated in the class diagram, the system's awareness is driven by a **GOAP::Blackboard** structure, which maps and maintains the current world state using generic **GOAP::Comparable** data types. The **GOAP::Planner** relies on this blackboard to evaluate its registered **GOAP::Goal** and **GOAP::Action** dictionaries.

To maintain parity between player and AI mechanics, the **GOAP::Action** class acts as an interface layer. Because these **GOAP::Action** objects explicitly define their requirements via **CheckPreconditions()** and **ApplyEffect()**, it allows the planner to utilise a backward-chaining search algorithm (A*) through the action state space.

Visual Debugger and Measure Success Criteria

To ensure the accuracy of the underlying simulation and to easily validate the game's spatial logic, a comprehensive visual debugger was implemented. Toggled via the Escape key during runtime, this overlay provides real-time visual insights into the game's hidden mechanics and data structures.

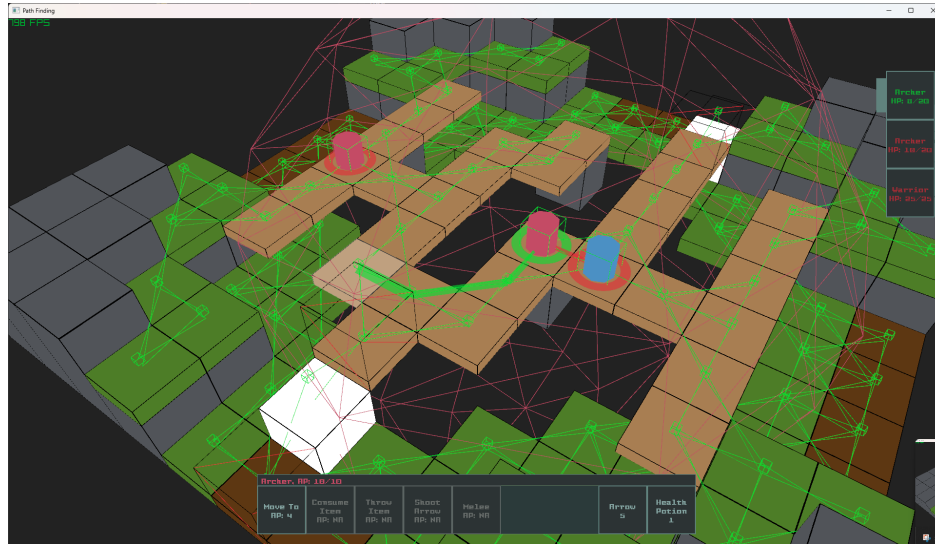


Figure 10: Visual debugger, currently its an archers turn, thus there is a pink spherical wireframe demonstrating the max shooting range

Navigation and Pathfinding Validation

The debugger visualises the underlying **SparseGraph** by rendering the individual nodes and their interconnecting edges across all traversable terrain. A critical feature of this system is the explicit representation of elevation changes. Sloped transitions are highlighted using a black wireframe cube, which demonstrates exactly where the higher and lower ground nodes connect. This visual proof is essential for verifying why diagonal pathing connections are restricted in these specific areas, preventing illogical movement or mesh clipping.

Character State and Combat Bounds

Combat parameters are also actively visualised to confirm they align with the initial game design specifications. For example, when an Archer becomes the active character, the debugger renders a spherical wireframe originating from their position. This dynamically illustrates their defined maximum shooting range (`m_MaxShootRange`) and helps verify LOS calculations. This ensures that both the player and the AI strictly adhering to the spatial constraints when formulating attacks.

Reflection and Integration

Task 19: Initial Architecture and Foundation

Overall, the development process has been highly rewarding, particularly in successfully implementing core systems such as the custom level parser, influence maps, and A* pathfinding. The design draws from a previous experience building a 2D engine in C# for COS20007 - Intro to OOP which made the architecture design and implementation easy. Surprisingly, A major technical roadblocks has not occurred yet, with no significant bugs requiring more than a few minutes to resolve.

The primary challenges encountered were structural and time allocation rather than the technical implementation (So far at least). Balancing the heavy concurrent workload of this unit and other subjects made allocating development time difficult. Furthermore, the core logic of the GOAP agent was clear, designing the diagrams (defining the connections, preconditions, effects) was very hard and required lots of iteration.

The final submission, implementing the GOAP system remains the largest hurdle, as I expect the integration and subsequent balancing of the AI's decision-making mechanics to require lots of testing and iteration.

Task 20: Final Integration and Scope Management Reflection

Reflecting on the final weeks of development, the primary challenge became time management and scope control. Coming off the high of building a complete platformer game from scratch for Introduction to Object Oriented Programming, I wanted to one-up myself with this custom project. However, this ambition led to an overly scoped feature set. The main lesson learnt is the absolute necessity of being stricter with project scoping from the outset.

As a result of this expanded scope, there ultimately wasn't enough time to complete the project in its entirety. The final build is missing the complete GOAP source implementation, although the foundational skeleton and architectural planning are intact. Integrating basic steering behaviours was a straightforward process that took very little time to implement effectively.